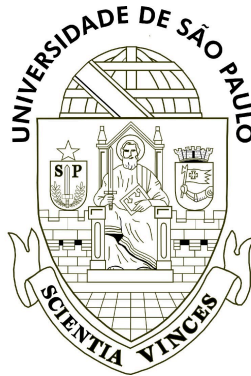


UNIVERSIDADE DE SÃO PAULO

Faculdade de Filosofia, Ciências e Letras de Ribeirão Preto

Departamento de Computação e Matemática



5954023 - COMPILADORES

Linter e Extrator de Métricas para miniLua

Trabalho 1: análise estática com Flex e Bison

INTEGRANTES

Francisco Eduardo Fontenele Ramos Neto (15452569)

Guilherme Borges de Pádua Barbosa (15653045)

Vinicius Botte (15522900)

PROFESSOR

Prof. Dr. José Augusto Baranauskas

Ribeirão Preto
Outubro de 2026

1. Escolhas de projeto

O analisador tem quatro partes: o lexer em Flex (`src/lexer.l`), a gramática em Bison (`src/parser.y`), o estado da análise (`src/analyzer.cpp`) e o programa principal (`src/main.cpp`). Usamos C++11 para ter `std::string`, `std::vector` e `std::map` no estado, mas mantivemos o esqueleto C do Bison e o lexer C do Flex, usando só recursos que existem desde o Bison 3.0 e o Flex 2.5.35. A interface entre os dois é a de sempre: `yylex()`, `yylval` e `yylloc`.

Não há AST. As ações semânticas chamam funções do `analyzer`, que mantém a pilha de funções abertas, a pilha de blocos de controle e a tabela de símbolos. Como a `%union` do esqueleto C só aceita tipos triviais, os valores semânticos são inteiros: índices para tabelas de nomes, listas e referências guardadas no `analyzer`. A exceção é `ExpVal`, uma struct de dois inteiros que diz se a expressão é um literal numérico isolado e se é uma função anônima.

As posições vêm de `%locations`. O lexer atualiza `yylloc` em todo token pela macro `YY_USER_ACTION`. Um contador de linha global no lexer daria a linha errada nos alertas, porque quando o Bison reduz uma regra o lexer já pode ter lido o token seguinte, que às vezes está na linha de baixo. Com `@n` cada ação usa a posição do próprio símbolo.

2. Como a gramática controla o escopo das funções

O ponto central é abrir o contexto da função antes de o corpo ser analisado, para que as chamadas, os operadores lógicos e as estruturas de controle encontrados no corpo sejam atribuídos a ela. Isso é feito com uma ação no meio da regra, executada logo depois do nome e antes dos parâmetros. O fechamento acontece quando o `end` é reduzido em `funcbody`:

```
stat      : T_FUNCTION funcname      { lint::begin_function($2, @1.first_line); }
          | T_LOCAL T_FUNCTION T_NAME { declare_local_function(...); begin_function($3, ...); }
          funcbody
exp       : T_FUNCTION               { lint::begin_function(-1, @1.first_line); } /* anônima */
          funcbody                   { $$ = lint::exp_function(); }
funcbody : '(' params ')' block T_END { lint::end_function(@5.last_line); }
```

`begin_function` empilha a função; `end_function` desempilha. Toda métrica é registrada na função do topo da pilha, ou num escopo global quando a pilha está vazia. Assim, numa função aninhada, as chamadas feitas dentro dela ficam com ela, e as feitas depois do seu `end` voltam para a de fora. Uma chamada é registrada quando a regra `functioncall` é reduzida, depois dos argumentos; por isso em `pcall(function() g() end)` a chamada a `g` fica com a anônima e a chamada a `pcall` com a função externa.

As estruturas de controle seguem o mesmo padrão. Em `T_IF exp T_THEN { push_control("if") } block elseifs else_part T_END { pop_control() }` a ação do meio conta o nó de decisão e empilha o bloco no caminho de aninhamento da função atual. Se o caminho fica maior que o mais profundo já visto, ele é guardado; é daí que sai a cadeia função `-> for -> for -> if`. O `elseif` só soma na complexidade, sem empilhar, e `and` e `or` somam quando a regra binária é reduzida. Como a ação do meio vem depois da condição, um `and` na condição de um `if` é contado antes do `if`; isso não importa porque o relatório mostra as estruturas e os operadores lógicos em grupos separados.

Os blocos léxicos usam uma ação vazia no início de `block` que abre um escopo na tabela de símbolos. Parâmetros e variáveis de laço são colocados numa lista pendente e entram no escopo quando o bloco do corpo abre, de modo que não são visíveis nas expressões de limite do `for`. Em `local x = expr` a declaração vem depois da expressão, como no Lua. Essa tabela é usada pelos alertas de variável não usada e de atribuição a global, e para não confundir parâmetro com variável global.

A gramática não tem conflitos e o arquivo declara `%expect 0`. Lua tem uma ambiguidade conhecida: em `a = f` seguido de uma linha que começa com `(`, o parêntese pode ser argumento de `f` ou o início de outro comando. Fizemos o comando de chamada ser `stat : prefixexp` (com verificação na ação de que é mesmo uma chamada), o que transforma o conflito redução/redução em deslocamento/redução, e demos a `(` precedência maior que as reduções de `prefixexp`. O resultado é o mesmo do Lua 5.2: a chamada continua.

3. Métricas e alertas

Linhas. O lexer conta todo `\n` que passa por ele, inclusive dentro de comentários e strings, e marca como lógica toda linha coberta por um token. Comentários nunca geram token, então linhas só com comentário ou em branco não são lógicas; uma string longa de três linhas marca as três.

Comandos múltiplos. A lista de comandos de um bloco carrega como valor semântico a última linha do comando anterior. Se o próximo comando começa nessa mesma linha, o alerta é emitido, uma vez por linha.

Magic numbers. Cada literal é registrado pelo lexer. Quando o parser reduz uma atribuição, declaração `local` ou campo de tabela, os literais que formam sozinhos uma expressão do lado direito (inclusive com sinal de menos) são marcados como isentos. Literais comparados com `==` ou `~=` também ficam isentos no modo padrão. No fim, os que sobram e não valem 0 nem 1 viram alerta, com o valor comparado numericamente para que `0x0` ou `1.0` também sejam aceitos.

Relatório. Os alertas são ordenados por linha. Cada função mostra parâmetros, chamadas externas (sem as recursivas), profundidade e complexidade $v(G) = 1 + N_d$ com o detalhamento.

4. Decisões de interpretação

- Linhas físicas são o número de `\n`, como no `wc -l`; é a única forma de obter as 44 linhas do teste3, cujo arquivo original não terminava em quebra de linha.
- O enunciado dá 11 linhas lógicas para o teste2, mas o código tem 12 linhas com código. Mantivemos 12.
- Acrescentamos `Funções Definidas: N`, já que a contagem é pedida no item I.1 e os exemplos não a mostram.
- O teste2 não aponta o 2 de `tipo_cliente == 2`, enquanto o teste3 aponta o 18 de `idade < 18`. Isentamos literais comparados por igualdade no modo padrão; o `--extra` aponta esses também.
- `repeat` soma 1 na profundidade como os outros laços; `elseif`, `else` e `do ... end` não somam. Função aninhada tem contagem própria.
- No detalhamento da complexidade, as estruturas aparecem na ordem da primeira ocorrência, depois `and` e depois `or`, ordem que reproduz os quatro exemplos.
- Recursão, inclusive `self:m()` dentro de `Classe:m`, não conta como chamada externa.

Com essas escolhas, a saída dos três exemplos é igual à do enunciado, exceto pela linha de contagem de funções e pelas 12 linhas lógicas do teste2.

5. Testes

O diretório `tests/cases` tem 23 arquivos `.lua`, cada um com um `.out` que guarda a saída padrão, a saída de erro e o código de saída, nos modos normal e `--extra`. O `make test` roda todos e mostra o diff de quem falhar. Além dos três exemplos, há casos para comentários de bloco com e sem nível, `---[[` (que é comentário de linha), `--` e `;` dentro de strings, todas as sequências de escape (incluindo `\z`, `\x`, `\ddd` e `\u{}`), strings longas, CRLF, arquivo sem quebra de linha final, arquivo vazio e só com comentários, números hexadecimais e em notação científica, funções aninhadas, anônimas, locais e métodos, `repeat`, código no escopo global, comandos múltiplos, os alertas extras e seis erros léxicos e sintáticos. Também compilamos com `-D_GLIBCXX_DEBUG` para checar acessos fora dos limites nos contêineres e rodamos os testes nesse modo. O pacote foi descompactado numa pasta limpa, compilado e testado de novo antes da entrega.

Exemplos do enunciado. A tabela compara a saída do linter com a esperada nos três exemplos. Todos os outros campos (parâmetros, chamadas, profundidade, complexidade com o detalhamento e todos os alertas) são idênticos.

Exemplo	Diferença em relação ao enunciado
teste1	só a linha Funções Definidas: 1
teste2	Funções Definidas: 1 e 12 linhas lógicas em vez de 11
teste3	só a linha Funções Definidas: 3

Saída real para o teste3, que exercita quase todas as métricas:

```

=== RELATÓRIO DE ANÁLISE ===
Arquivo: teste3.lua
Linhas Físicas: 44
Linhas Lógicas: 37
Funções Definidas: 3

--- Função: valida_usuario ---
Parâmetros: 3 (idade, status, pontuacao)
Chamadas de Funções Externas: 2 (registra_evento, registra_evento)
Profundidade Máxima de Aninhamento: 2
Complexidade Ciclomática: 6 (1 base + 2 if + 1 elseif + 1 and + 1 or)

--- Função: processa_matriz ---
Parâmetros: 2 (linhas, colunas)
Chamadas de Funções Externas: 2 (cria_matriz_vazia, log_operacao)
Profundidade Máxima de Aninhamento: 4 (função -> for -> for -> if)
Complexidade Ciclomática: 4 (1 base + 2 for + 1 if)

--- Função: busca_limite ---
Parâmetros: 2 (valores, limite_maximo)
Chamadas de Funções Externas: 1 (alerta_limite)
Profundidade Máxima de Aninhamento: 3 (função -> while -> if)
Complexidade Ciclomática: 3 (1 base + 1 while + 1 if)

--- Alertas do Linter ---
[Aviso] Linha 3: 'Magic Number' detectado (18). Considere extrair para uma constante.
[Aviso] Linha 7: 'Magic Number' detectado (1000). Considere extrair para uma constante.
[Aviso] Linha 10: 'Magic Number' detectado (500). Considere extrair para uma constante.
[Aviso] Linha 27: Múltiplos comandos na mesma linha detectados.
[Aviso] Linha 40: 'Magic Number' detectado (999). Considere extrair para uma constante.
=====

```

Outros casos. Alguns trechos dos testes adicionais e o que o linter produz para eles:

Entrada	Resultado
local function interna(x) return transforma(x) end dentro de externa	transforma conta para interna, não para externa
ordena(lista, function(a, b) return compara(a, b) end)	função (anônima, linha 29) com a chamada a compara; ordena fica com a função de fora
function Conta:saca(v) ... self:saca(0) end	chamada recursiva, não conta como externa
repeat ... if ... while ... end end until t >= 3 or ...	profundidade 4 (função -> repeat -> if -> while), complexidade 5
local a = "texto com -- e ;"	nenhum comentário nem comando múltiplo detectado
--[[fechado]] local fator = 7	a linha conta como lógica; o comentário não
x * 0xFF, x + 1e3, x + 0x0	alerta para 0xFF e 1e3; 0x0 vale 0 e é aceito
if a then return x end numa linha	sem alerta; if a then x = y; y = x end gera alerta
função sem o end do if	erro de sintaxe: fim do arquivo inesperado; a função da linha 1 não foi fechada com 'end'
local s = "sem fim	erro léxico: string não terminada, com linha e coluna

6. Limitações

- Não há recuperação de erro: o primeiro erro léxico ou sintático interrompe a análise do arquivo e nenhum relatório é impresso para ele.
- Ficaram de fora recursos de Lua que não estão no miniLua: `goto` e rótulos, operadores bit a bit e atributos como `<const>`.
- Um literal entre parênteses, como `local x = (5)`, é apontado como magic number, porque a isenção só vale para o literal sozinho.
- O reconhecimento de recursão é por nome. Uma função guardada em outra variável e chamada por ela conta como externa.
- Colunas nas mensagens de erro são contadas em bytes, então caracteres acentuados antes do erro deslocam a coluna.
- Arquivos que usam só `\r` como fim de linha (Mac clássico) são lidos como uma linha só.

7. Extras

- Números hexadecimais (`0xFF`, `0x1p4`) e em notação científica (`1e3`, `2.5E-2`), com erro para números malformados como `12abc`.
- Opção `--extra` com alertas de função longa (mais de 50 linhas lógicas), mais de 5 parâmetros, aninhamento acima de 4, complexidade acima de 10, variável, parâmetro ou função local não usados, atribuição a global dentro de função e arquivo sem quebra de linha final. Ficam atrás da opção para não mudar a saída dos exemplos.
- Mensagens de erro com arquivo, linha e coluna, em português. Quando falta um `end`, a mensagem diz qual bloco ficou aberto e em que linha.
- Sintaxe de Lua além do miniLua: tabelas, indexação, métodos, `for ... in`, `local function`, funções anônimas (nomeadas pela variável ou campo que as recebe), strings longas e comentários com nível.
- Vários arquivos por execução, leitura da entrada padrão e seção de métricas para o código fora de funções.

8. Divisão do trabalho

Integrante	Responsabilidade
Francisco Eduardo Fontenele Ramos Neto (15452569)	Analisador léxico, contagem de linhas, literais e strings
Guilherme Borges de Pádua Barbosa (15653045)	Gramática, controle de escopo das funções e blocos, mensagens de erro
Vinicius Botte (15522900)	Métricas, alertas, relatório, testes e documentação